



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Macroarea di Ingegneria
Dipartimento di Ingegneria Civile e Ingegneria Informatica

Predizione di Classi Buggy con Machine Learning

Studio di fattibilità su tecniche per migliorare il supporto al testing

Corso di Ingegneria del Software 2
Laurea Magistrale in Ingegneria Informatica

A.A. 2024/25
Matteo Basili

Agenda

- Introduzione e Obiettivi
- Raccolta Dati e Labeling
- Addestramento e Valutazione dei Modelli
- Risultati
- Conclusioni

Introduzione

- Il **software testing** è una fase fondamentale per garantire qualità e affidabilità, ma risulta spesso *costosa* e *time-consuming*



- Individuare in anticipo le parti più problematiche può ridurre significativamente sforzi e tempi di verifica

☞ Adottare un approccio **risk-based** e **predittivo** permette di ottimizzare l'utilizzo delle risorse

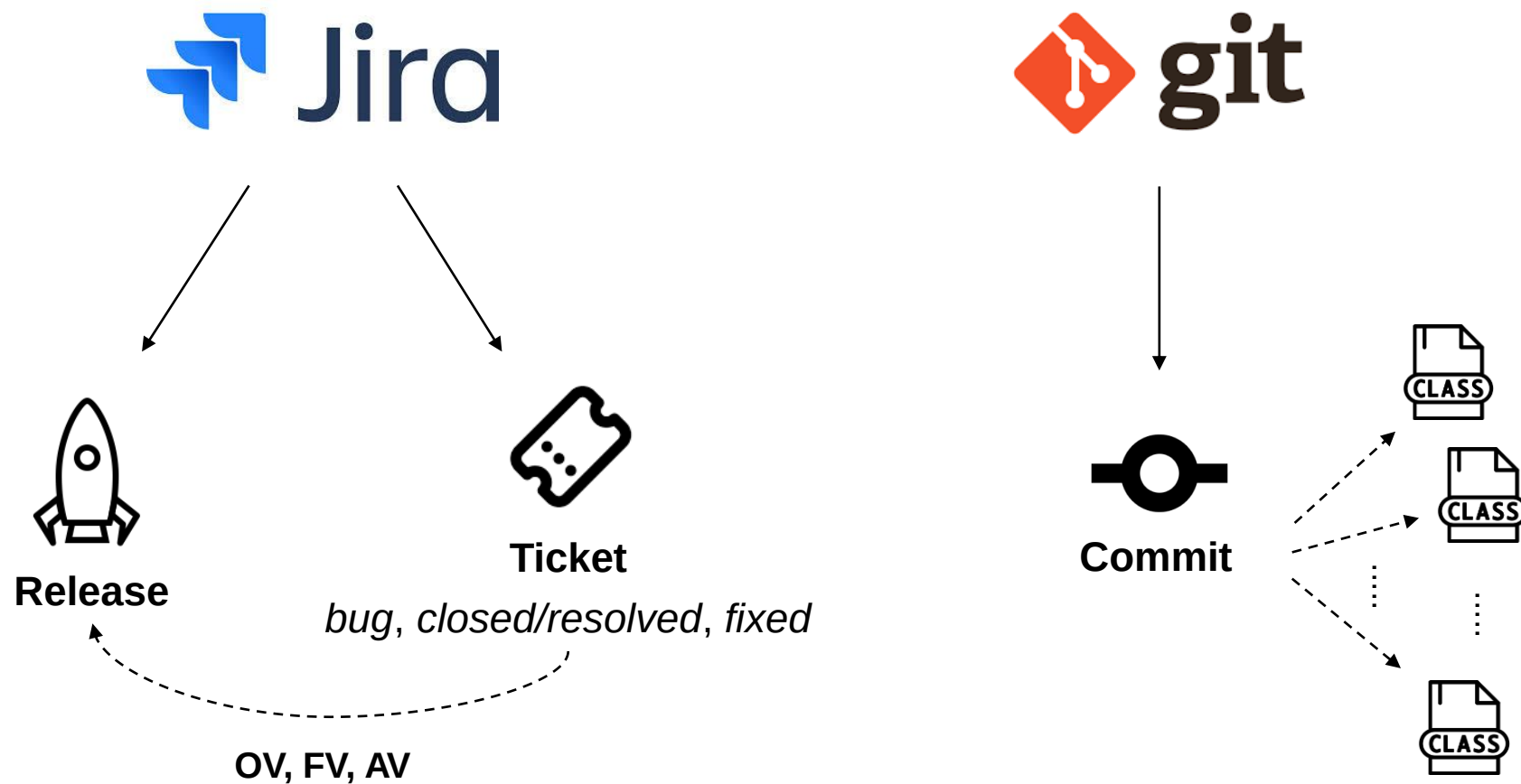
Obiettivi

- Valutare la **fattibilità** dell'uso di modelli di **Machine Learning (ML)** per predire classi **buggy**
- Rendere il processo di testing **più efficiente**, concentrando gli sforzi sulle classi a maggior rischio
- Costruire modelli ML in grado di rispondere alla domanda:
“Questa classe sarà buggy nel prossimo rilascio?”
- Analizzare diverse configurazioni
 - **Feature selection**
 - **Balancing**
 - **Cost-sensitive**
- Eseguire uno studio empirico su 2 progetti open-source
 - **Apache BookKeeper**
 - **Apache Storm**

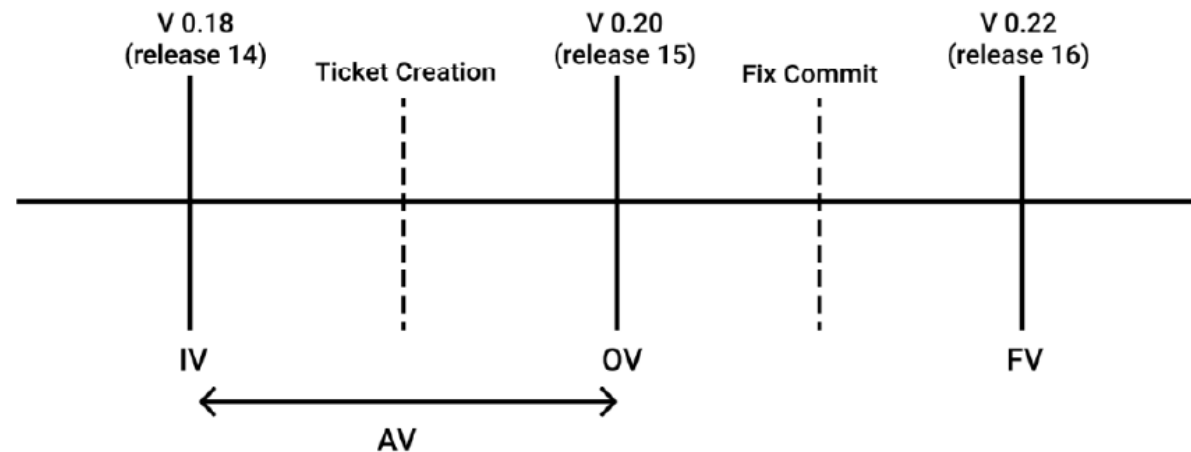
Assunzioni

- **Il futuro è simile al passato**
 - Il comportamento evolutivo del software segue pattern appresi dai dati storici
- **Linearità delle release**
 - Si assume un ciclo evolutivo lineare, senza rami paralleli multipli presi in analisi
- **Jira come fonte completa dei ticket**
 - Si assume che tutti i bug siano tracciati in *Jira*

Raccolta Dati



Ciclo di Vita di un Difetto



- Il difetto inizialmente non esiste
- In una versione (non sempre nota) viene introdotto → **Injected Version (IV)**
- Il difetto viene scoperto solo nella versione in cui viene aperto il ticket → **Opening Version (OV)**
- La prima release senza il difetto è la **Fixed Version (FV)**
- Tutte le versioni impattate dal difetto si chiamano **Affected Versions (AV)** → $IV = \min(AV)$

Labeling

- L'obiettivo è stimare la IV, quando non è esplicitamente riportata nel ticket
→ **Proportion (Increment)**
- Per i ticket con IV nota, si calcola la proporzione

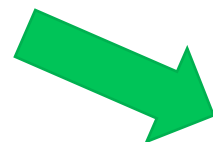
$$P = \frac{FV - IV}{FV - OV}$$

- Il valore P viene mantenuto e aggiornato incrementalmente, come media dei valori dei ticket precedenti con IV conosciuta
- Per ogni ticket senza IV → $IV_{stimata} = FV - (FV - OV) * P$
- Una classe è considerata difettosa se toccata da un commit del bug in una versione compresa tra IV e FV

Metriche Estratte

- Metriche di **classe**
- Calcolate per ogni release

Variabile da predire



Metrica	Descrizione
Size	Linee di codice
LOC Added	Linee di codice aggiunte
Average LOC Added	Media delle LOC aggiunte per revisione
Max LOC Added	Massimo numero di LOC aggiunte in una singola revisione
LOC Removed	Linee di codice rimosse
Average LOC Removed	Media delle LOC rimosse per revisione
Max LOC Removed	Massimo numero di LOC rimosse in una singola revisione
Churn	Differenza tra LOC aggiunte e rimosse
Average Churn	Media del churn per revisione
Max Churn	Churn più alto in una singola revisione
NR	Numero di revisioni
Nfix	Numero di fix di difetti
Nauth	Numero di autori
Bugginess	Difettosità

Addestramento & Valutazione

- Vengono utilizzati modelli di Machine Learning **supervisionati**
- Il dataset viene suddiviso
 - **Training Set** → per apprendere il modello
 - **Testing Set** → per valutarne le performance
- Poiché i dati evolvono nel tempo, si utilizza una validazione **walk-forward**
- Per limitare lo **snoring**, si considerano solo le prime metà delle release

Run	Part				
	1	2	3	4	5
1	Testing				
2	Training	Testing			
3	Training	Training	Testing		
4	Training	Training	Training	Testing	
5	Training	Training	Training	Training	Testing

Variabile Target

- L'obiettivo è stabilire, per ogni release, se una classe è **buggy** o meno
- Alla i -esima release:

- **Training Set**

- Deve risultare realistico
 - Etichettatura basata su ticket con $FV \leq (i-1)$
 - Dai commit associati ai ticket si estraggono le classi modificate
 - Le classi modificate vengono etichettate come buggy
 - Può soffrire di snoring

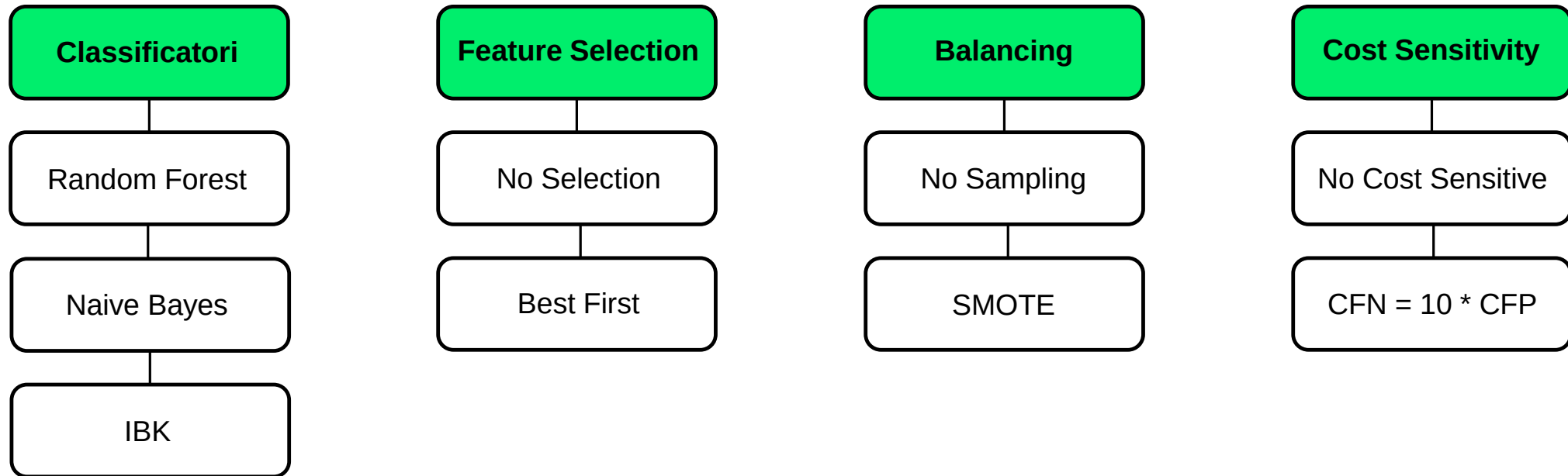


- **Testing Set**

- Deve essere il più corretto possibile
 - Etichettatura basata su tutti i ticket disponibili fino al momento attuale (fino alla n -esima release)



Modelli e Tecniche

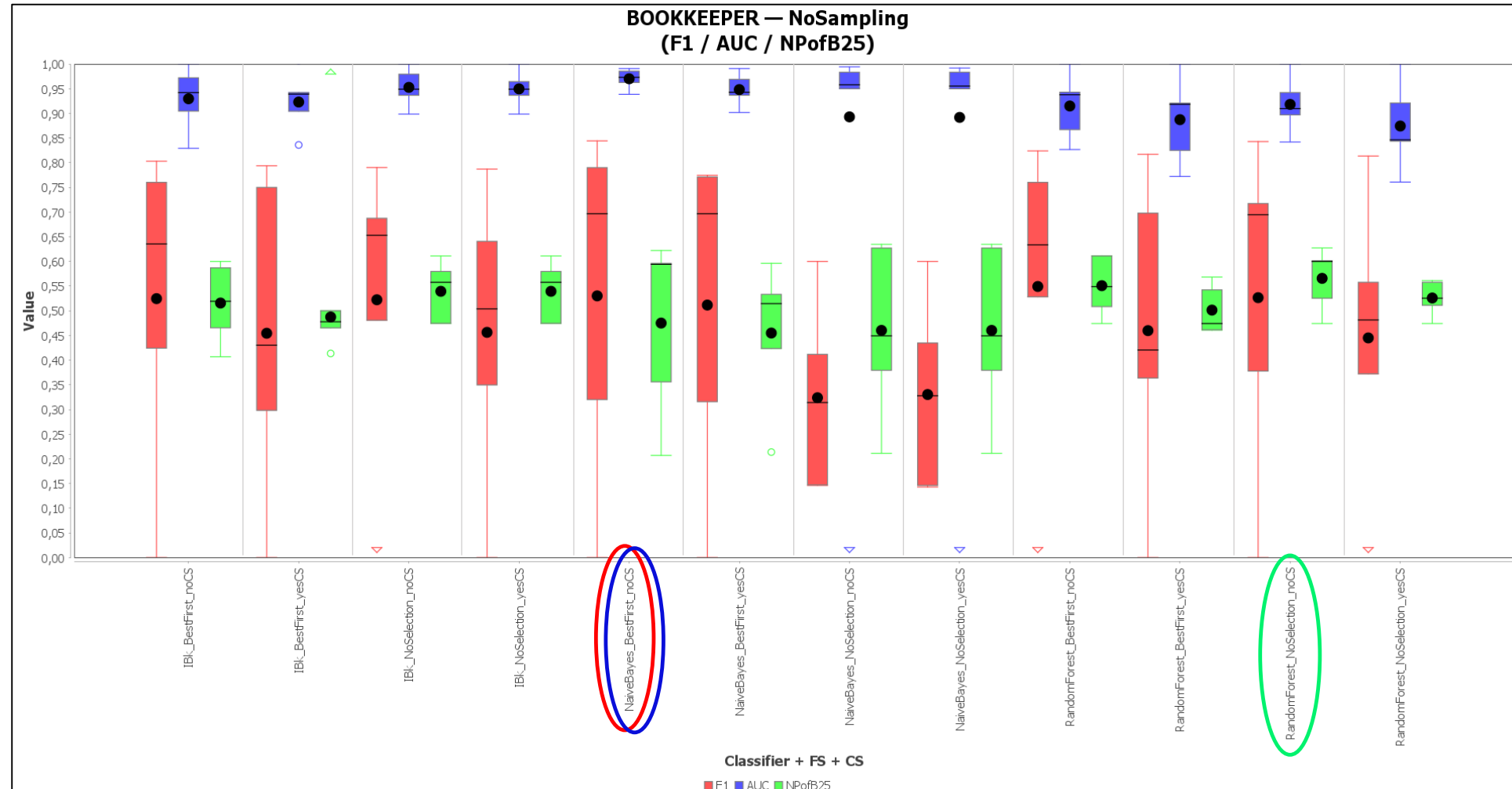


- Sono state valutate tutte le possibili combinazioni

Valutazione

- Metriche utilizzate per ogni classificatore:
 - **F1** = $2 * \frac{Precision * Recall}{Precision + Recall}$
 - **AUC** – probabilità che un esempio positivo scelto a caso sia classificato con punteggio più alto di un esempio negativo scelto a caso
 - **MCC** = $\frac{TP * TN - FP * FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$
 - **NPofB25** – percentuale di bug individuati analizzando il 25% del codice più “a rischio”, considerando la probabilità di difettosità normalizzata per la dimensione delle entità
- Calcolate tramite **ACUME** (<https://github.com/jonidacarka/ACUME>)

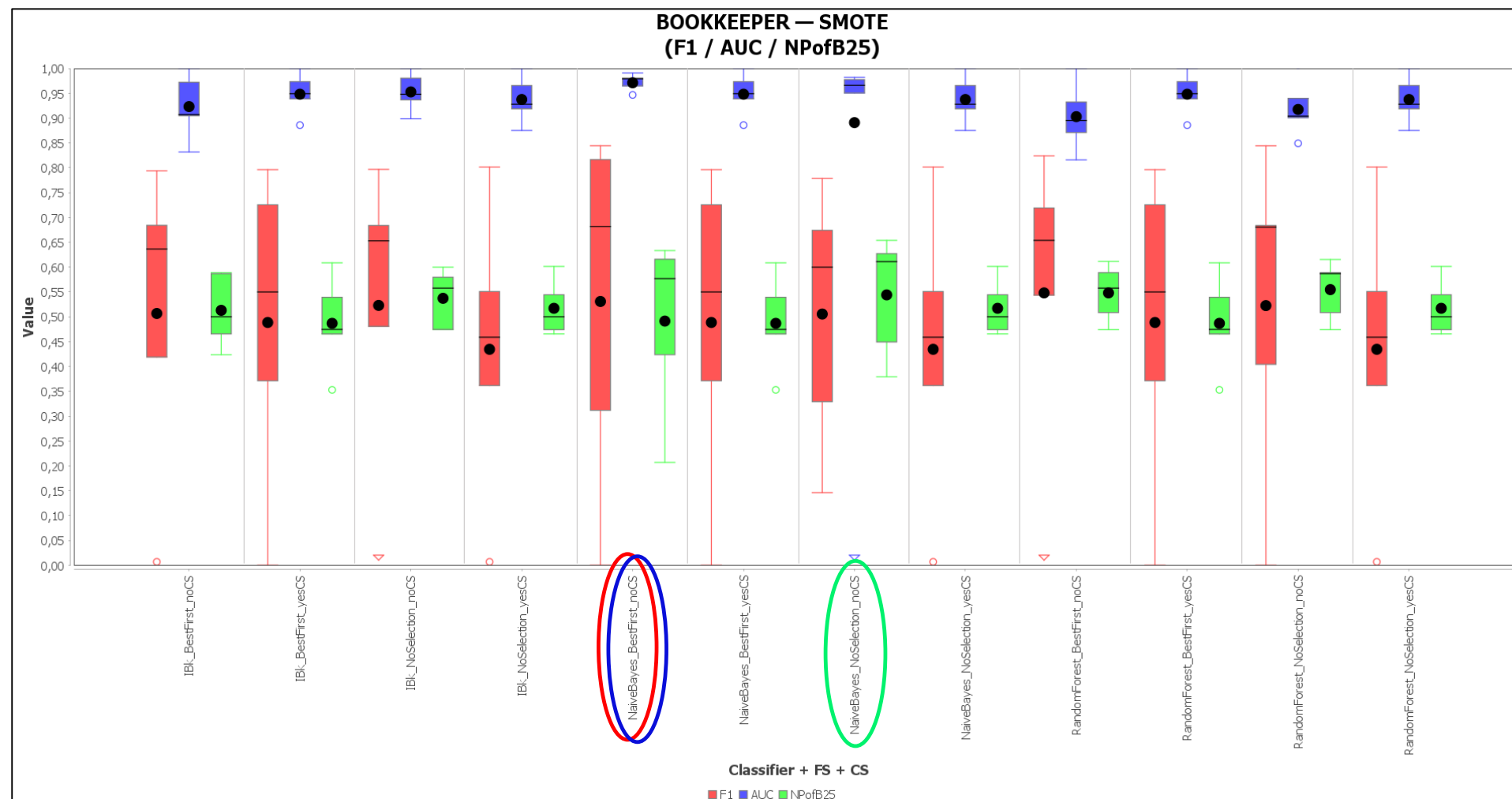
Risultati – BOOKKEEPER



Risultati – BOOKKEEPER (2)



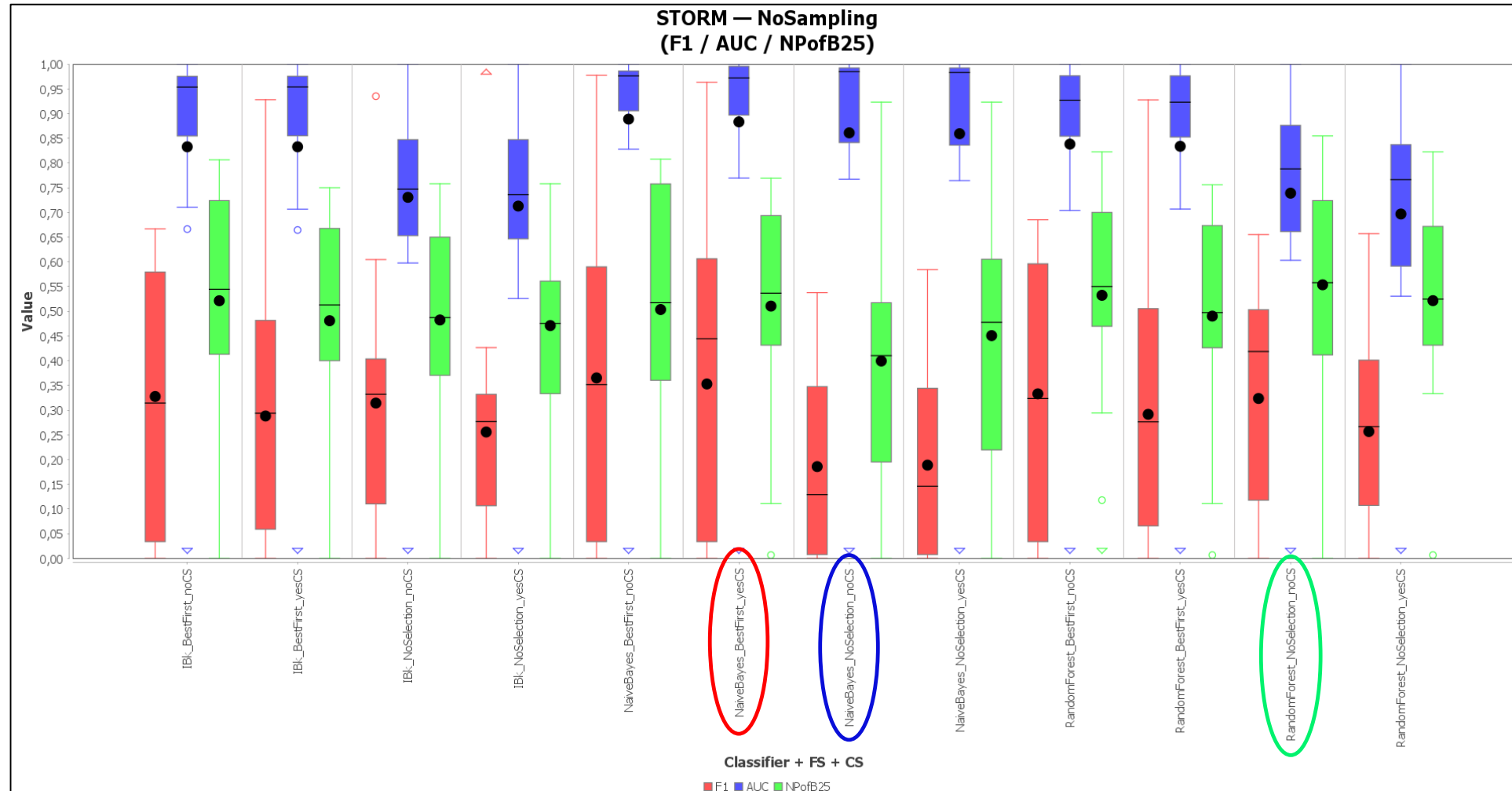
Risultati – BOOKKEEPER (3)



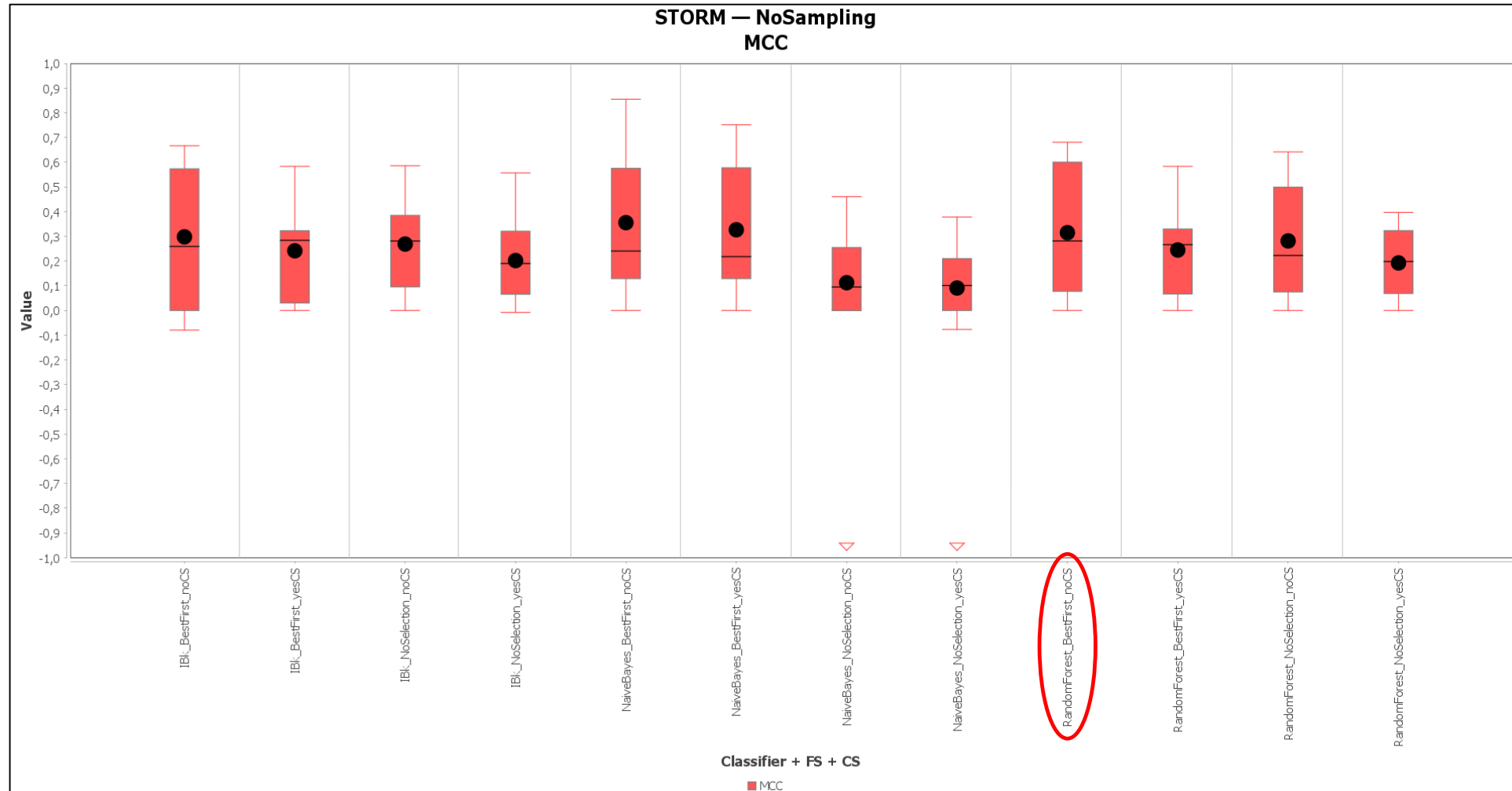
Risultati – BOOKKEEPER (4)



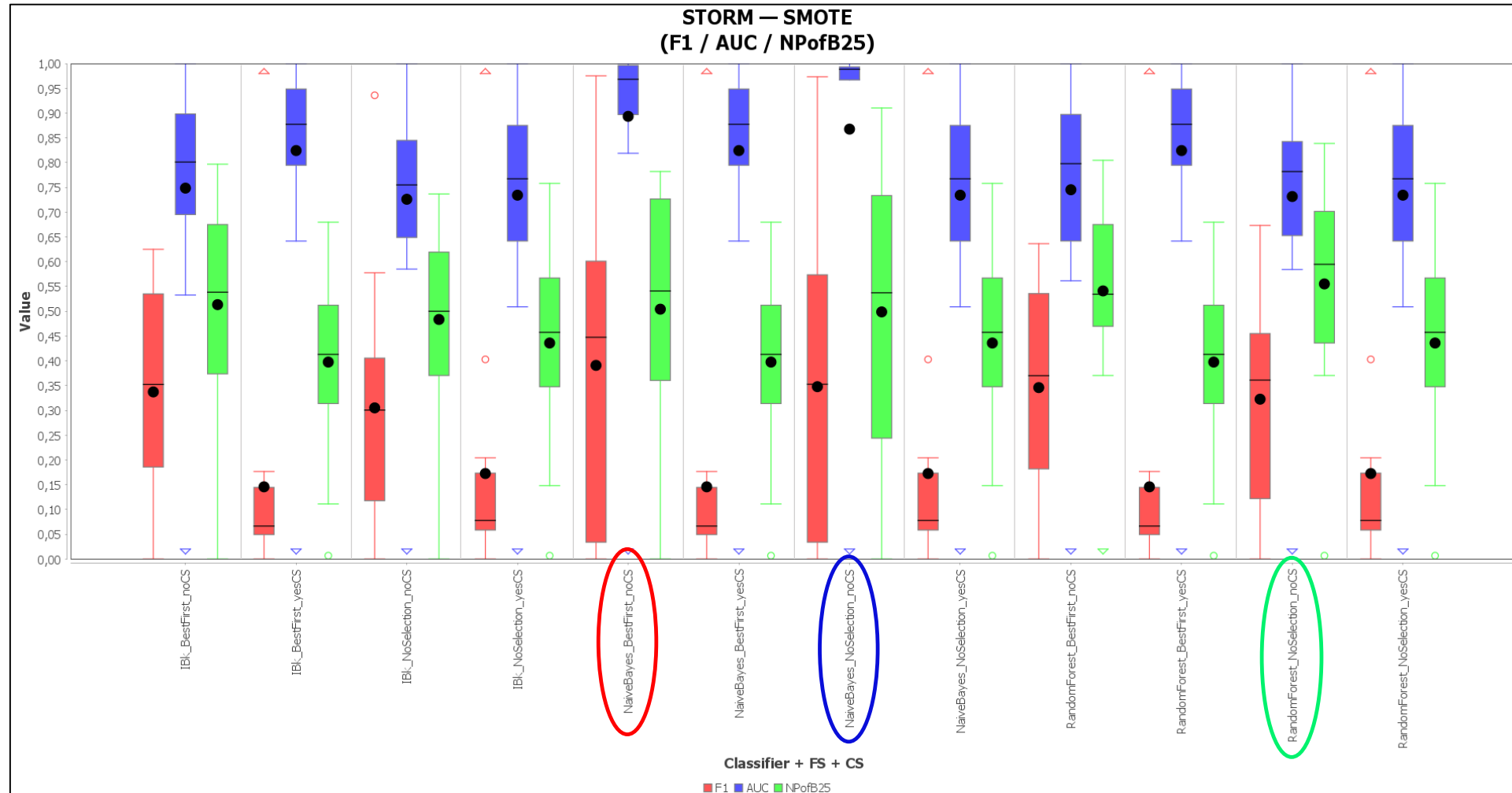
Risultati – STORM



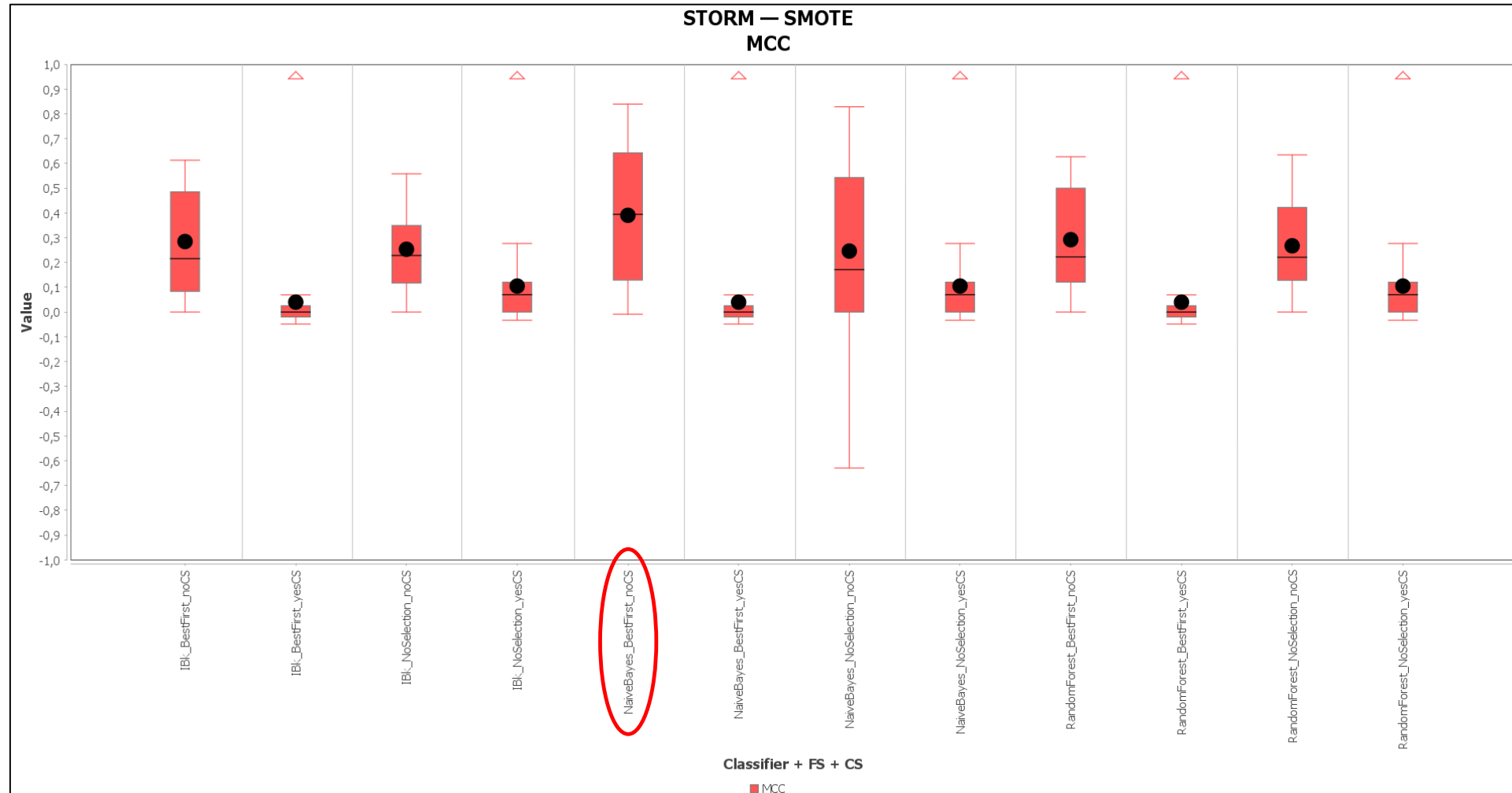
Risultati – STORM (2)



Risultati – STORM (3)



Risultati – STORM (4)



Conclusioni

“La feature selection, il balancing e la cost sensitivity migliorano l’accuratezza della bug prediction?”

- **Naïve Bayes** è il classificatore più efficace per metriche di accuratezza globale
 - Migliori risultati su F1, AUC e MCC nella maggior parte delle configurazioni
 - Feature selection (Best First) migliora sistematicamente le prestazioni
 - Cost sensitivity ha impatto limitato e non sistematico
- **Random Forest** emerge sulle metriche *effort-aware*
 - Migliori valori di NPofB25 su entrambi i progetti
 - Le migliori configurazioni non utilizzano feature selection né cost sensitivity
- **Balancing (SMOTE)**
 - Non modifica in modo sostanziale il ranking dei classificatori
 - I migliori risultati sono spesso ottenuti anche senza sampling

☞ Le tecniche analizzate migliorano l’accuratezza solo in combinazioni specifiche e dipendenti dal progetto e dalla metrica

Conclusioni (2)

“Quale configurazione dovrebbe adottare un project manager?”

- Accuratezza globale (F1, AUC, MCC)
 - ✓ Naïve Bayes + Best First Feature Selection
 - ✓ Configurazione efficace su BOOKKEEPER e STORM
 - ✗ Cost sensitivity e SMOTE non necessari
- Supporto alla fase di testing (NPofB25)
 - ✓ Random Forest senza feature selection e senza cost sensitivity
 - ✓ Migliori risultati su entrambi i progetti
- BOOKKEEPER
 - Comportamento più stabile
 - Naïve Bayes domina tutte le metriche eccetto NPofB25
- STORM
 - Maggiore eterogeneità
 - Richiede valutazione per metrica



L'adozione di modelli di bug prediction è fattibile, ma deve essere guidata dagli obiettivi (*accuratezza vs testing*) e calibrata sul progetto target

Minacce alla Validità e Limiti

- Possibili errori nella mappatura ticket–commit → etichette non sempre affidabili
- Le metriche usate potrebbero non riflettere davvero la “bugginess”
- Fattori esterni (team, esperienza, pratiche) possono influenzare i risultati
- I repository analizzati potrebbero non rappresentare altri progetti software
- Dataset potenzialmente piccoli → risultati non sempre statisticamente significativi
- L’approccio Increment adottato è conservativo → possibile sottostima delle performance
- Dipendenza da strumenti (Jira, Git) → errori o limiti degli strumenti possono ridurre la riproducibilità
- Date di creazione/risoluzione ticket non sempre precise
- Ticket etichettati “bug” potrebbero non esserlo realmente

Riferimenti

- **Github**



- [*https://github.com/MatteoBasili/isw2-progetto-project_management-2024_25*](https://github.com/MatteoBasili/isw2-progetto-project_management-2024_25)

- **SonarCloud**



- [*https://sonarcloud.io/project/overview?id=MatteoBasili_isw2-progetto-project_management-2024_25*](https://sonarcloud.io/project/overview?id=MatteoBasili_isw2-progetto-project_management-2024_25)